

Phase 2 Test Specifications

POC-01 — Loan Application Management System

Total Test Cases: 20 | **Pass Threshold:** 14 of 20 (70%)

INGESTION TESTS (4 cases)

TC-01-P2-ING-01: User Manual Loads Successfully

Category: Ingestion | **Priority:** High **Prerequisites:** `rag/user_manual.md` file exists with content

Steps:

1. Call `TextLoader("rag/user_manual.md").load()`
2. Verify at least 1 document returned

Pass Criteria: `len(documents) >= 1`; no exception raised; content is non-empty

```
def test_document_loads_successfully():
    from langchain_community.document_loaders import TextLoader
    loader = TextLoader("rag/user_manual.md", encoding="utf-8")
    documents = loader.load()
    assert len(documents) >= 1
    assert len(documents[0].page_content) > 100
```

TC-01-P2-ING-02: Chunking Produces Correct Sizes

Category: Ingestion | **Priority:** High **Prerequisites:** User manual loaded

Steps:

1. Load user manual
2. Split with `RecursiveCharacterTextSplitter(chunk_size=512, chunk_overlap=50)`
3. Verify chunk sizes

Pass Criteria: All chunks have `len(chunk.page_content) <= 560` (512 + small tolerance for splitter); at least 20 chunks created from the manual

```
def test_chunking_produces_correct_sizes():
    from langchain_community.document_loaders import TextLoader
    from langchain.text_splitter import RecursiveCharacterTextSplitter

    documents = TextLoader("rag/user_manual.md").load()
    splitter = RecursiveCharacterTextSplitter(chunk_size=512, chunk_overlap=50)
    chunks = splitter.split_documents(documents)
```

```
assert len(chunks) >= 20 # Manual should produce many chunks
for chunk in chunks:
    assert len(chunk.page_content) <= 600, f"Chunk too large:
{len(chunk.page_content)}"
```

TC-01-P2-ING-03: Embeddings Generated for Chunks

Category: Ingestion | **Priority:** High **Prerequisites:** `GOOGLE_API_KEY` set; chunks available

Steps:

1. Create embeddings for a single chunk text
2. Verify embedding is a non-empty list of floats

Pass Criteria: Embedding is a list with length > 0; all elements are floats

```
def test_embeddings_generated():
    import os
    from langchain_google_genai import GoogleGenerativeAIEmbeddings

    embeddings = GoogleGenerativeAIEmbeddings(
        model="models/text-embedding-004",
        google_api_key=os.getenv("GOOGLE_API_KEY")
    )

    test_text = "The minimum CIBIL score for a personal loan is 650."
    result = embeddings.embed_query(test_text)

    assert isinstance(result, list)
    assert len(result) > 0
    assert all(isinstance(x, float) for x in result)
```

TC-01-P2-ING-04: ChromaDB Persists Vectors

Category: Ingestion | **Priority:** High **Prerequisites:** Ingestion script has been run at least once

Steps:

1. Create `PersistentClient` pointing to `./chroma_db`
2. Get collection `poc_01_loan_manual`
3. Verify collection count > 0

Pass Criteria: Collection exists; `collection.count() > 0`

```
def test_chromadb_persists_vectors():
    import chromadb
```

```
client = chromadb.PersistentClient(path="./chroma_db")
collection = client.get_collection("poc_01_loan_manual")

assert collection is not None
count = collection.count()
assert count > 0, f"Collection empty – run ingest.py first"
```

RETRIEVAL TESTS (6 cases)

TC-01-P2-RET-01: Relevant Chunk Retrieved for Document Query

Category: Retrieval | **Priority:** High **Prerequisites:** ChromaDB populated with manual chunks

Steps:

1. Build retriever with `top_k=4`
2. Query: "What documents are required for a home loan?"
3. Verify at least 1 retrieved chunk contains loan/document-related content

Pass Criteria: Retrieved docs contain at least one of: "id_proof", "income_proof", "property_docs", "bank_statement"

```
def test_relevant_chunk_retrieved_for_doc_query():
    import os
    from langchain_google_genai import GoogleGenerativeAIEmbeddings
    from langchain_chroma import Chroma

    embeddings = GoogleGenerativeAIEmbeddings(
        model="models/text-embedding-004",
        google_api_key=os.getenv("GOOGLE_API_KEY")
    )
    vectorstore = Chroma(
        collection_name="poc_01_loan_manual",
        embedding_function=embeddings,
        persist_directory="./chroma_db"
    )
    retriever = vectorstore.as_retriever(search_kwargs={"k": 4})

    docs = retriever.invoke("What documents are required for a home loan?")

    combined_text = " ".join(doc.page_content.lower() for doc in docs)
    assert any(term in combined_text for term in ["id_proof", "income_proof",
"property", "bank_statement"])
```

TC-01-P2-RET-02: Top-K Returns Correct Number of Results

Category: Retrieval | **Priority:** High

Steps:

1. Query retriever with `top_k=4`
2. Count returned documents

Pass Criteria: Exactly 4 documents returned (or fewer if collection has < 4 docs)

```
def test_top_k_returns_k_results():  
    # ... setup retriever with k=4 ...  
    docs = retriever.invoke("loan application status")  
    assert len(docs) == 4
```

TC-01-P2-RET-03: Out-of-Scope Query Returns Low-Relevance Chunks

Category: Retrieval | **Priority:** Medium **Note:** Tests that the system doesn't confidently retrieve irrelevant content

Steps:

1. Query retriever with: "What is the best recipe for pasta carbonara?"
2. Verify no retrieved chunk is highly relevant to the query

Pass Criteria: Retrieved chunks do not contain "pasta", "recipe", "carbonara"; the generation step should handle this gracefully

```
def test_irrelevant_query_low_similarity():  
    # The retriever will still return chunks (ChromaDB always returns top-k)  
    # but the generation should say it doesn't have that info  
    docs = retriever.invoke("What is the best recipe for pasta carbonara?")  
  
    combined = " ".join(doc.page_content.lower() for doc in docs)  
    # Chunks should be loan-related, not pasta-related  
    assert "pasta" not in combined  
    assert "recipe" not in combined
```

TC-01-P2-RET-04: Retrieval for Credit Score Query

Category: Retrieval | **Priority:** High

Steps:

1. Query: "What is the minimum credit score for a home loan?"
2. Verify retrieved chunks mention credit score

Pass Criteria: At least 1 retrieved chunk contains "credit score" or "CIBIL"

```
def test_retrieval_for_credit_score_query():
    docs = retriever.invoke("What is the minimum credit score for a home loan?")
    combined = " ".join(doc.page_content.lower() for doc in docs)
    assert "credit" in combined or "cibil" in combined
```

TC-01-P2-RET-05: Empty Query Handled Gracefully

Category: Retrieval | **Priority:** Medium

Steps:

1. Query retriever with empty string ""

Pass Criteria: No exception raised; returns 0 or k documents

```
def test_empty_query_handled():
    try:
        docs = retriever.invoke("")
        # Should not raise, may return empty or default results
        assert isinstance(docs, list)
    except Exception as e:
        pytest.fail(f"Empty query raised exception: {e}")
```

TC-01-P2-RET-06: Retrieval Latency Under Threshold

Category: Retrieval | **Priority:** Medium

Steps:

1. Measure time to retrieve docs for a standard query
2. Verify under 5 seconds

Pass Criteria: Retrieval completes in < 5 seconds

```
def test_retrieval_latency_under_threshold():
    import time
    start = time.time()
    docs = retriever.invoke("How long does loan approval take?")
    elapsed = time.time() - start
    assert elapsed < 5.0, f"Retrieval took {elapsed:.2f}s, expected < 5s"
```

GENERATION TESTS (6 cases)

TC-01-P2-GEN-01: Answer Grounded in Context

Category: Generation | **Priority:** High **Prerequisites:** Full RAG chain built and ChromaDB populated

Steps:

1. Query: "What is the minimum CIBIL score for a personal loan?"
2. Verify answer mentions "650"

Pass Criteria: Answer string contains "650"

```
def test_answer_grounded_in_context():
    from rag.rag_chain import build_rag_chain
    chain, retriever = build_rag_chain()

    answer = chain.invoke("What is the minimum CIBIL score for a personal loan?")

    assert isinstance(answer, str)
    assert len(answer) > 10
    assert "650" in answer
```

TC-01-P2-GEN-02: Correct Document Requirements Listed

Category: Generation | **Priority:** High

Steps:

1. Query: "What documents are required for a home loan?"
2. Verify answer mentions all required document types

Pass Criteria: Answer contains at least 3 of: "id_proof", "income_proof", "bank_statement", "property"

```
def test_answer_references_correct_doc_type():
    answer = chain.invoke("What documents are required for a home loan?")
    answer_lower = answer.lower()

    required_terms = ["id", "income", "bank_statement", "property"]
    found = [term for term in required_terms if term in answer_lower]
    assert len(found) >= 3, f"Answer missing key documents: {answer}"
```

TC-01-P2-GEN-03: FAQ Question Answered Correctly

Category: Generation | **Priority:** High

Steps:

1. Query: "How long does the personal loan approval process take?"
2. Verify answer mentions 2-3 days or similar timeframe

Pass Criteria: Answer contains a timeframe ("day", "week", "business day")

```
def test_answer_for_faq_question():
    answer = chain.invoke("How long does the personal loan approval process
take?")
    assert any(word in answer.lower() for word in ["day", "week", "business",
"hours"])
```

TC-01-P2-GEN-04: No Hallucination on Out-of-Scope Query

Category: Generation | **Priority:** High

Steps:

1. Query: "What is the interest rate for gold loans?"
2. Verify answer indicates information not available (gold loans not in manual)

Pass Criteria: Answer contains phrases like "don't have information", "not available", "contact", or "helpdesk"

```
def test_no_hallucination_on_out_of_scope():
    answer = chain.invoke("What is the interest rate for gold loans?")
    out_of_scope_indicators = [
        "don't have", "not available", "contact", "helpdesk",
        "not in", "unable to", "no information"
    ]
    assert any(phrase in answer.lower() for phrase in out_of_scope_indicators), \
        f"Chatbot may have hallucinated: {answer}"
```

TC-01-P2-GEN-05: Response Format Is a String

Category: Generation | **Priority:** High

Steps:

1. Query any valid question
2. Verify response is a non-empty string

Pass Criteria: `isinstance(answer, str)` and `len(answer) > 0`

```
def test_response_format_is_string():
    answer = chain.invoke("What are the age requirements for a personal loan?")
    assert isinstance(answer, str)
    assert len(answer.strip()) > 0
```

TC-01-P2-GEN-06: Query About Rejection Process

Category: Generation | **Priority:** Medium

Steps:

1. Query: "What happens if my loan application is rejected?"
2. Verify answer mentions key points from manual (90 days, new application)

Pass Criteria: Answer mentions "new application" or "90 days" or "rejection reason"

```
def test_answer_about_rejection_process():
    answer = chain.invoke("What happens if my loan application is rejected?")
    assert any(phrase in answer.lower() for phrase in
               ["new application", "90 day", "rejection reason", "reason",
                "reapply"])
```

OBSERVABILITY TESTS (4 cases)

TC-01-P2-OBS-01: LangSmith Trace Created

Category: Observability | **Priority:** High **Prerequisites:** LANGCHAIN_TRACING_V2=true, LANGCHAIN_API_KEY set

Steps:

1. Run a RAG query
2. Check LangSmith API for recent traces in project **AI-Readiness-POC-01-P2**

Pass Criteria: At least 1 run exists in the LangSmith project after querying

```
def test_langsmith_trace_created():
    import os
    from langsmith import Client

    # Run a query first
    chain.invoke("What documents are needed for a personal loan?")

    # Check LangSmith
    client = Client(api_key=os.getenv("LANGCHAIN_API_KEY"))
    runs = list(client.list_runs(
        project_name="AI-Readiness-POC-01-P2",
        limit=5
    ))
    assert len(runs) > 0, "No LangSmith traces found"
```

TC-01-P2-OBS-02: OTEL Spans Exported for Ingestion

Category: Observability | **Priority:** High **Prerequisites:** OTel console exporter configured; ingestion script has `tracer.start_as_current_span` calls

Steps:

1. Capture stdout while running ingestion
2. Verify span names appear in output

Pass Criteria: Console output contains "rag.document_load", "rag.chunk", and "rag.embed"

```
def test_otel_spans_exported_during_ingestion(capsys):
    from rag.ingest import ingest_manual
    ingest_manual("rag/user_manual.md")

    captured = capsys.readouterr()
    # OTel console exporter outputs to stdout
    # Check span names appeared
    assert "rag.document_load" in captured.out or "document_load" in captured.out
```

TC-01-P2-OBS-03: Structured Log Fields Present

Category: Observability | **Priority:** High

Steps:

1. Run a RAG query
2. Capture log output
3. Parse JSON log lines

Pass Criteria: Log lines contain `poc_id`, `phase`, `operation` fields

```
def test_log_fields_present(capsys):
    import json

    from rag.rag_chain import build_rag_chain, answer_question
    chain, retriever = build_rag_chain()
    answer_question("Test query", chain, retriever)

    captured = capsys.readouterr()
    # Try to parse log lines as JSON
    for line in captured.out.split('\n'):
        try:
            log_entry = json.loads(line)
            if log_entry.get("operation") == "question_answered":
                assert "poc_id" in log_entry
                assert "phase" in log_entry
                assert log_entry["poc_id"] == "POC-01"
                return # Test passes if we find the right log entry
        except json.JSONDecodeError:
```

```
continue
```

```
# If we reach here, no matching log entry found
pytest.fail("No structured log entry with poc_id and phase found")
```

TC-01-P2-OBS-04: Trace Contains Retrieval Metadata

Category: Observability | **Priority:** Medium **Prerequisites:** LangSmith tracing enabled

Steps:

1. Run a query with `@traceable` decorator
2. Fetch the trace from LangSmith API
3. Verify trace has retrieval information

Pass Criteria: Trace exists in LangSmith with `run_type` that includes retrieval step

```
def test_trace_contains_retrieval_metadata():
    import os, time
    from langsmith import Client

    chain.invoke("What is the maximum personal loan amount?")
    time.sleep(3) # Wait for trace to be sent

    client = Client(api_key=os.getenv("LANGCHAIN_API_KEY"))
    runs = list(client.list_runs(
        project_name="AI-Readiness-POC-01-P2",
        limit=1,
        run_type="chain"
    ))

    assert len(runs) > 0
    run = runs[0]
    assert run.name is not None
```

Running Phase 2 Tests

```
# Ensure Phase 2 is set up first
python rag/ingest.py # Run ingestion before tests

# Run all Phase 2 tests
pytest tests/phase2/ -v

# Run specific category
pytest tests/phase2/ -v -k "ING" # Ingestion tests
pytest tests/phase2/ -v -k "RET" # Retrieval tests
pytest tests/phase2/ -v -k "GEN" # Generation tests
```

```
pytest tests/phase2/ -v -k "OBS"    # Observability tests

# Generate report
pytest tests/phase2/ --junitxml=results/phase2-results.xml
```

Important Notes for Phase 2 Testing

1. **Always run `ingest.py` before running tests** — generation and retrieval tests require populated ChromaDB
2. **Generation tests use real LLM calls** — they incur API usage and may take 2-5 seconds each
3. **Observability tests require real LangSmith credentials** — skip if no API key available (use `pytest -k "not OBS"`)
4. **Retry flaky tests** — LLM responses can vary; if a generation test fails once, re-run before concluding failure